

Informe XML Schema

Javier Ortín Rodenas

Contenido

Informe XML Schema	1
Cambios en tipos de datos	2
Tipos de datos predefinidos	2
Fecha de la carrera	2
Hora de la carrera	2
Número de vueltas.....	2
Longitud del circuito	2
Anchura media	3
Tiempo de victoria.....	3
Puntos de cada clasificado	3
Tipos personalizados con restricciones	4
Tipo puesto	4
Tipo distancia	4
Tipo longitud.....	4
Tipo latitud	5
Tipo altitud	5
Restricciones de cardinalidad	5
Referencias.....	5
Galería de fotografías	6
Vídeos del circuito.....	6
Podio de clasificados.....	6
Libertad en el orden	7
Elemento raíz	7
Componentes de las coordenadas	8
Orden de los atributos de los tramos	8

Cambios en tipos de datos

Tipos de datos predefinidos

En el DTD, todos los elementos de texto se definían genéricamente como `#PCDATA`. En muchos casos, podemos mejorar esta validación al usar tipos predefinidos de XML que se adapten mejor a la semántica del campo que representan.

Fecha de la carrera

Usamos el tipo `xs:date` de la siguiente forma:

```
<xs:element name="fecha" type = "xs:date"/>
```

Nos permite asegurarnos de que la fecha sigue un formato YYYY-MM-DD bien estructurado.

Hora de la carrera

Análogamente, usamos el tipo `xs:time` como sigue:

```
<xs:element name="hora" type="xs:time"/>
```

Número de vueltas

Ahora usamos el tipo `xs:positiveInteger` para que no sean negativas:

```
<xs:element name="vueltas" type="xs:positiveInteger"/>
```

Además, nos aseguramos así de que sea un número entero.

Longitud del circuito

Usamos también `xs:positiveInteger` como en el caso anterior. Seguimos manteniendo las unidades como `string`:

```
<xs:element name="longitud-circuito">
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="xs:positiveInteger">
        <xs:attribute name="unidades" type="xs:string" use="required" />
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>
```

Anchura media

Seguimos el mismo procedimiento que con la longitud del circuito:

```
<xs:element name="anchura-media">
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="xs:positiveInteger">
        <xs:attribute name="unidades" type="xs:string" use="required" />
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>
```

Tiempo de victoria

Al tratarse del tiempo transcurrido entre dos instantes, usamos el tipo de datos `xs:duration` de la siguiente forma:

```
<xs:element name="tiempo-victoria" type="xs:duration" />
```

Ahora las unidades vienen dadas por el propio tipo duración, luego no es necesario mantener el atributo “unidades” del DTD.

Puntos de cada clasificado

Los modelamos como enteros positivos:

```
<xs:element name="clasificado">
  <xs:complexType>
    <xs:simpleContent>
      <xs:extension base="xs:string">
        <xs:attribute name="puesto" type="tipoPuesto" use="required" />
        <xs:attribute name="puntos" type="xs:positiveInteger" use="required" />
      </xs:extension>
    </xs:simpleContent>
  </xs:complexType>
</xs:element>
```

El tipo `tipoPuesto` se explicará a continuación.

Tipos personalizados con restricciones

De manera similar al caso anterior, cambiamos el tipo base de partida a uno más adecuado, pero introducimos además restricciones sobre el rango de valores posible que pueden tomar.

Tipo puesto

Para medir los puestos de los clasificados:

```
<xs:simpleType name="tipoPuesto">
  <xs:restriction base="xs:positiveInteger">
    <xs:minInclusive value="1"/>
    <xs:maxInclusive value="3"/>
  </xs:restriction>
</xs:simpleType>
```

Tipo distancia

Usamos un rango de valores plausible para la longitud de los tramos, que puede tener decimales:

```
<xs:simpleType name="tipoDistancia">
  <xs:restriction base="xs:decimal">
    <xs:minExclusive value="0"/>
    <xs:maxInclusive value="500"/>
  </xs:restriction>
</xs:simpleType>
```

Tipo longitud

De manera similar, acotamos la longitud de coordenadas geográficas:

```
<xs:simpleType name="tipoLongitud">
  <xs:restriction base="xs:decimal">
    <xs:minInclusive value="-180"/>
    <xs:maxInclusive value="180"/>
  </xs:restriction>
</xs:simpleType>
```

Tipo latitud

Análogo al caso anterior, pero para la latitud:

```
<xs:simpleType name="tipoLatitud">
  <xs:restriction base="xs:decimal">
    <xs:minInclusive value="-90"/>
    <xs:maxInclusive value="90"/>
  </xs:restriction>
</xs:simpleType>
```

Tipo altitud

Mide la elevación sobre el nivel del mar de los puntos del circuito:

```
<xs:simpleType name="tipoAltitud">
  <xs:restriction base="xs:decimal">
    <xs:minExclusive value="0"/>
    <xs:maxInclusive value="1000"/>
  </xs:restriction>
</xs:simpleType>
```

Restricciones de cardinalidad

Indican el número mínimo o máximo de elementos que puede tener un tipo compuesto (como `all` o `sequence`).

Referencias

Tres o más referencias:

```
<xs:element name="referencias">
  <xs:complexType>
    <xs:sequence>
      <xs:element minOccurs="3" maxOccurs="unbounded" ref="referencia" />
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Galería de fotografías

Mínimo una fotografía, máximo cinco:

```
<xs:element name="fotografias">
  <xs:complexType>
    <xs:sequence>
      <xs:element minOccurs="1" maxOccurs="5" ref="fotografia" />
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Vídeos del circuito

Mínimo unos y máximo tres:

```
<xs:element name="videos">
  <xs:complexType>
    <xs:sequence>
      <xs:element minOccurs="1" maxOccurs="3" ref="video" />
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Podio de clasificados

Mostramos el podio al completo, pero nada más:

```
<xs:element name="clasificados">
  <xs:complexType>
    <xs:sequence>
      <xs:element minOccurs="3" maxOccurs="3" ref="clasificado" />
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Libertad en el orden

Se ha cambiado sequence por all en todos los tipos compuestos heterogéneos para hacer más flexible el documento sin renunciar a robustez de validación. Al procesar el xml se ha de tener en cuenta el tipo de datos y usar las expresiones XPath adecuadas. Por tanto, el orden en tipos heterogéneos no debería ser un problema.

Elemento raíz

No importa el orden de los campos del elemento raíz:

```
<xs:element name="circuito">
  <xs:complexType>
    <xs:all>
      <xs:element ref="nombre"/>
      <xs:element ref="longitud-circuito" />
      <xs:element ref="localidad"/>
      <xs:element ref="pais" />
      <xs:element ref="anchura-media" />
      <xs:element ref="fecha" />
      <xs:element ref="hora" />
      <xs:element ref="vueltas" />
      <xs:element ref="patrocinador" />
      <xs:element ref="referencias" />
      <xs:element ref="fotografias" />
      <xs:element ref="videos" />
      <xs:element ref="origen" />
      <xs:element ref="tramos" />
      <xs:element ref="vencedor" />
      <xs:element ref="clasificados" />
    </xs:all>
  </xs:complexType>
```

Componentes de las coordenadas

Ahora las componentes podrán ir en cualquier orden:

```
<xs:element name="coordenadas">
  <xs:complexType>
    <xs:all>
      <xs:element ref="longitud" />
      <xs:element ref="latitud" />
      <xs:element ref="altitud" />
    </xs:all>
  </xs:complexType>
</xs:element>
```

Se aplica tanto para el origen como para los tramos del circuito.

Orden de los atributos de los tramos

Ahora cuentan con más flexibilidad:

```
<xs:element name="tramo">
  <xs:complexType>
    <xs:all>
      <xs:element ref="distancia" />
      <xs:element ref="coordenadas" />
    </xs:all>
    <xs:attribute name="sector" type="xs:string" use="required" />
  </xs:complexType>
</xs:element>
```